Enhancing Flight Deck Decision Support with Distributed GenAI: A Multi-Agent Approach

Jose Pacheco De Almeida Prado, David Zammit Mangion, Brian Zammit, Jason Gauci
University of Malta
Msida, Malta, MSD 2080
jose.pachecodealmeida@um.edu.mt
david.zammit-mangion@um.edu.mt
brian.zammit@um.edu.mt
jason.gauci@um.edu.mt

Alan Muscat, Sandro Mizzi, Andre Manicolo
QuAero Ltd.
Malta
alan.p.muscat@quaero.aero
sandro.mizzi@quaero.aero
andre.manicolo@quaero.aero

Abstract— **The integration of Generative Artificial Intelligence (GenAI) into commercial aviation presents transformative opportunities for enhancing flight deck operations, offering an intuitive natural language interface between pilots and automation. This study investigates a novel GenAI-based multi-agent architecture designed to address the unique demands of aviation, such as the need for rapid decision-making and adherence to stringent safety protocols. By employing lightweight, embedded Large Language Models (LLMs), the architecture optimises task allocation among specialised agents, ensuring operational efficiency without reliance on external cloud infrastructure.**

Preliminary evaluations demonstrate that the proposed architecture achieves performance comparable to systems using larger models, such as GPT-4, while operating locally with lightweight models. This result underscores the feasibility of implementing autonomous, cloud-independent GenAI solutions embedded directly within aircraft systems. Through a comparative analysis of different LLM configurations, the system balances scalability and precision in handling cockpit-specific tasks. Challenges related to explainability, response latency, and integration with broader Human-Machine Interface (HMI) systems are identified as critical areas for future development.

TABLE OF CONTENTS

1. INTRODUCTION 1
2. LITERATURE REVIEW 3
3. OVERVIEW OF THE PROPOSED ARCHITECTURE 7
4. EXPERIMENTAL TESTING AND RESULTS 10
5. DISCUSSION AND CONCLUSION 15
ACKNOWLEDGEMENTS 16
REFERENCES 16
BIOGRAPHY 19

1 INTRODUCTION

Generative Artificial Intelligence (GenAI) has emerged as a transformative field within Artificial Intelligence (AI), enabling the creation of new content, such as text, images, and sounds, rather than merely analyzing existing data. GenAI models, including Large Language Models (LLMs), are trained on vast datasets and exhibit capabilities resembling human reasoning, such as question answering and complex content generation [1]. These capabilities have significant implications across various industries, particularly in commercial aviation and the flight deck environment. In our view, the integration of GenAI into the cockpit represents a novel opportunity for significant advancements in decision support, real-time monitoring, and incident/event management. Potential applications include scenarios such as en-route diversions, weather avoidance, fuel management, and flight path optimization.

The cockpit environment of commercial aircraft is highly complex, requiring pilots to make quick decisions based on multiple variables and considerations. To ensure safety and efficiency while minimising the risk of human error, crew support systems must rapidly generate data-driven solutions. GenAI, acting as an intuitive interface between pilots and the aircraft's systems—including onboard automation—provides real-time assistance, enhancing situational awareness and reducing workload [2]. By complementing traditional Human-Machine Interaction (HMI) methods, such as touch screens and keypads, with natural language interfaces via voice or text, GenAI fosters a more adaptive and human-centric interaction framework. This additional layer of interaction allows pilots to focus on critical tasks while maintaining seamless communication with cockpit systems.

GenAI's role extends beyond basic data retrieval, facilitating access to both static information—such as operation manuals—and dynamic, real-time data, including weather conditions, traffic surveillance, and airport operational status. By integrating with Machine Learning (ML) systems, GenAI enhances cockpit operations with predictive insights, such as anticipating weather disruptions or optimizing flight routes. This synergy provides pilots with actionable information tailored to the current context, improving decision-making efficiency and operational safety.

Figure 1 illustrates the central role of GenAI as an intermediary between pilots, onboard automation, and various internal and external data sources. By processing pilot inputs through natural language commands—via voice or text—GenAI retrieves relevant information from systems such as the Flight Management System (FMS), aircraft manuals, and weather databases. This information is then

2025 IEEE Aerospace Conference | 979-8-3503-5597-0/25/$31.00 ©2025 IEEE | DOI: 10.1109/AERO63441.2025.11068478

979-8-3503-5597-0/25/$31.00 ©2025 IEEE

Authorized licensed use limited to: ULAKBIM UASL - IZMIR YUKSEK TEKNOLOJI ENSTITUSU. Downloaded on February 01,2026 at 23:53:35 UTC from IEEE Xplore. Restrictions apply.

presented back to the pilot through formats that align with the operational context, including text displays, synthetic voice, or existing visual interfaces. Rather than replacing traditional HMI methods, GenAI complements them, enhancing the flexibility of cockpit operations and enabling pilots to switch seamlessly between multiple input and output modalities depending on workload and situational demands.

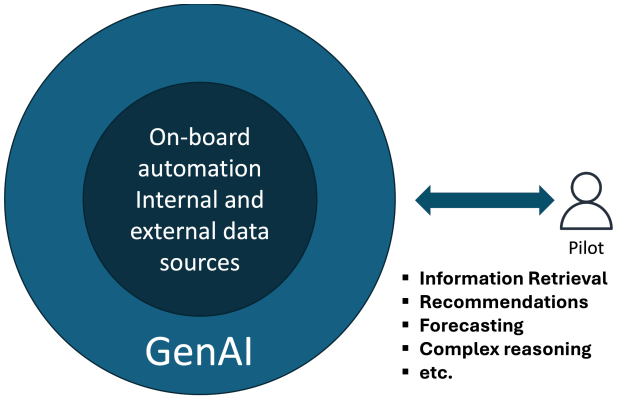


Figure 1. GenAI provides a natural language interface between humans, automation, and various data sources.

While GenAI has demonstrated remarkable versatility in fields such as data analysis and content generation, its application in aviation, particularly within the cockpit, introduces unique and critical challenges. Aviation requires solutions that adhere to stringent safety regulations and operational standards, addressing the need to process both structured data—such as weather and sensor readings—and unstructured data, including technical manuals, incident reports, and voice communications. Ensuring that GenAI systems can seamlessly handle these diverse data types is pivotal to their reliability and effectiveness in high-stakes environments like the flight deck.

Furthermore, aviation operates within a highly technical framework, characterised by specific terminology, intricate procedures, and stringent regulatory standards. While general-purpose GenAI models, such as OpenAI's GPT-4, excel in many Natural Language Processing (NLP) tasks, they require significant modifications to address aviation-specific challenges. These include interpreting complex terminology, adapting to specialised operational contexts, and ensuring full compliance with industry safety standards. For instance, cockpit-focused GenAI must accurately respond to nuanced queries about aircraft systems and operations, while providing outputs that align with regulatory and operational requirements.

Another significant barrier to implementing GenAI in the cockpit is response time and latency. Aviation demands rapid and accurate decision-making, even in off-nominal or unpredictable scenarios. General-purpose LLMs, such as GPT-4, are resource-intensive and often exhibit slower response times, rendering them unsuitable for real-time applications during flight. Additionally, their reliance on static datasets and lack of real-time data integration further limits their effectiveness in the dynamic environment of the flight deck.

To address these challenges, specialised models such as AviationGPT [3] and aeroBERT-Classifier [4] have been developed and tailored to aviation-specific tasks, including the interpretation of technical manuals and air traffic forecasting. Research by Yucheng et al. [5] highlights the importance of GenAI solutions capable of processing aviation-specific data and performing complex reasoning tasks, such as route planning. Expanding on this, Andriuškevičius et al. [6] proposed a multi-agent architecture that leverages different LLMs working in coordination to resolve air traffic conflicts without human intervention. Integrating such systems into the cockpit is crucial to ensuring the reliability, accuracy, and usability of AI-provided information by pilots. Furthermore, GenAI systems must offer clear explanations and justifications for their recommendations, particularly in safety-critical environments.

An additional aspect to consider is the regulation and certification requirements imposed on specific aircraft categories by authorities such as the Federal Aviation Administration (FAA) and the European Union Aviation Safety Agency (EASA). Certification may necessitate adopting specific design approaches, such as ensuring autonomous operation without relying on external cloud infrastructure and incorporating built-in fault tolerance to prevent operational interruptions during flight. Pilots must retain control, with sufficient visibility to review, understand, and reject AI-generated suggestions when necessary. Compliance with guidelines from EASA (and other regulatory bodies) on AI trustworthiness, explainability, transparency, robustness, human-AI teaming (HAT), and safety will be essential [7].

Currently, no formal process exists for certifying AI systems in civil aviation cockpits. However, efforts like the 'EASA Artificial Intelligence Roadmap 2.0' [7] are laying the groundwork for standardizing AI applications more broadly in aviation. Despite these advancements, the certification of AI-based systems remains a challenge, as conventional software certification processes cannot be directly applied to non-deterministic AI systems. Unlike traditional systems, AI applications often involve probabilistic behaviors and rely heavily on training datasets, making it difficult to ensure predictability and robustness under critical operational conditions. New assurance methodologies, such as EASA's 'W-shaped learning assurance process' [8], are being explored to address these issues, but a universally accepted framework is yet to be established.

In response to these challenges, this paper investigates the design and implementation of a GenAI architecture tailored to commercial aircraft cockpits. The proposed architecture employs a distributed multi-agent framework, integrating relatively small, specialised models optimised for cockpit-specific tasks. At this stage of the project, the focus lies on

Authorized licensed use limited to: ULAKBIM UASL - IZMIR YUKSEK TEKNOLOJI ENSTITUSU. Downloaded on February 01,2026 at 23:53:35 UTC from IEEE Xplore. Restrictions apply.

evaluating the architecture's foundational capabilities—such as task decomposition, agent coordination, and response accuracy—without yet addressing how it interfaces with the cockpit's HMI, such as touch panels or physical controls. By concentrating on the underlying system, this work lays the groundwork for future integration efforts, where the architecture will support intuitive and seamless pilot interactions within a fully realised cockpit environment.

The remainder of the paper is structured as follows: Section 2 reviews developments in LLMs and their applicability to aviation, highlighting key advancements and limitations. Section 3 introduces the proposed architecture, detailing its components and their respective roles within the system. Section 4 describes the experimental setup and evaluates the performance of the models and agents. Finally, Section 5 discusses the findings, identifies limitations, and proposes directions for future research.

2. LITERATURE REVIEW

Large Language Models

Until 2017, Language Models (LMs) were primarily based on Recurrent Neural Networks (RNNs) [9] and their variants, such as Long Short-Term Memory (LSTM) Networks [10] and Gated Recurrent Units (GRUs) [11]. These models were trained for specific Natural Language Processing (NLP) tasks, including translation, sentiment analysis, summarisation, and question answering. Each of these tasks required a distinct neural network model.

The introduction of the Transformer architecture in the paper "Attention Is All You Need" [12] in 2017 marked a breakthrough. By leveraging the self-attention mechanism it allowed language models to process contextual dependencies in text more effectively, thereby eliminating the need for RNNs. This innovation improved the understanding of word meaning based on context, enhancing performance in NLP tasks.

Transformers also enabled the development of Pre-trained Language Models (PLMs) [1], which follow the "pre-training and fine-tuning" paradigm. In pre-training, models learn general language patterns from a large corpora of unlabelled text. Subsequently, they are fine-tuned on smaller datasets for specific tasks. For example, a model is first pre-trained to understand English and then fine-tuned for sentiment analysis or text summarisation, something that was not feasible with RNN-based models.

Early models such as BERT (Bidirectional Encoder Representations from Transformers) [13] and GPT (Generative Pre-trained Transformer) [14] adopted the Transformer architecture, but with different approaches: BERT uses a bidirectional approach, meaning it considers the context from both the left and right sides of a token sequence during training, which makes it suitable for tasks such as text classification, question answering and named entity recognition, where a deep understanding of the input context is required. In comparison, GPT adopts a unidirectional approach, where the context is considered only from the left side, making it suitable for generative tasks. BERT and GPT demonstrated the architecture's potential, resulting in subsequent versions, such as GPT-2 [15] and BART (Bidirectional and Auto-Regressive Transformers) [16] that expanded capabilities for complex NLP tasks by combining BERT and GPT techniques. BART, for example, enables both bidirectional encoding and auto-regressive decoding, meaning it uses bidirectional attention to understand the full context of the input (like BERT), while generating output in a sequential manner, one token at a time (like GPT), which makes it effective for both text generation and understanding tasks.

Continuous advances in PLMs led to the development of LLMs. Researchers discovered that increasing model size and training data resulted in improved performance on complex tasks, as described by the 'scaling law' in [17]. LLMs such as GPT-3, with 175 billion parameters, and PaLM, with 540 billion, exemplify this trend. These larger models began to exhibit emergent abilities—skills not observed in smaller models such as BERT (330 million parameters) or GPT-2 (1.5 billion parameters) and not predictable from their performance. According to Wei [18], emergent abilities are capabilities that only manifest at a certain scale of model size and training data. Examples of GPT-3's emergent abilities include language generation, in-context learning, and enhanced world knowledge.

The realisation of emergent properties has significantly influenced the growth and adoption of LLMs, making them widely accessible. As a result, the term "Large Language Model" is now commonly used to describe any model using the Transformer approach, while the term "Pre-trained Language Model" remains more academic. Recent studies [19, 20] indicate that emergent abilities are not exclusive to models with hundreds of billions of parameters; models with only a few billion parameters may also exhibit these skills, depending on the training process and dataset. One particularly important emergent ability in the aviation context is the capacity to reason through complex problems by decomposing tasks and planning the required steps—an area of key interest for further research. This skill is critical for aviation scenarios, such as flight route planning or emergency response, where the ability to accurately decompose and solve complex problems can significantly impact safety and efficiency.

Apart from emergent abilities, the ability of Transformers to understand complex patterns has made LLMs applicable across various domains, including other data types such as images, audio, and video. However, LLMs trained on vast amounts of static data face some limitations, namely:

- **Limited knowledge transparency:** It is challenging to determine precisely what knowledge is intrinsically stored in the model. For example, some models may recognise what a Meteorological

Authorized licensed use limited to: ULAKBIM UASL - IZMIR YUKSEK TEKNOLOJI ENSTITUSU. Downloaded on February 01,2026 at 23:53:35 UTC from IEEE Xplore. Restrictions apply.

Aerodrome Report (METAR) is, while others might not.

- **Outdated knowledge:** LLMs lack awareness of events that occurred after their training.
- **Inability to access dynamic data:** They cannot provide real-time information, such as the current temperature at specific locations.
- **Hallucination:** LLMs can generate incorrect information or fabricate false narratives to simply produce a response for the user.
- **Response length and structure:** Some models tend to produce overly long answers or struggle to maintain a structured format as requested by the user.

These limitations are critical in contexts like aviation, where correct and reliable information is necessary. Hallucinations, for instance, could mislead the crews in critical decision-making scenarios. Although PLMs and LLMs are applicable across various fields, careful system design is essential to mitigate these risks. Mechanisms need to be in place to detect and prevent issues like hallucinations to ensure that the LLMs can be trusted in high-stake and safety-critical environments.

Prompts, Prompt Engineering and In-Context Learning

In GenAI, a prompt is the input provided to a model to generate the desired output. Depending on the model and application, this output can range from answering questions and summarising documents to generating images or even producing audio files. The prompt is intended to guide the model's response, helping it understand context and produce relevant and coherent outputs.

While good results can be achieved with simple prompts containing only a basic question, as demonstrated by Brown et al. [21], the quality of the response depends on how much contextual information is embedded in the prompt and how well it is structured. A prompt may therefore include, in addition to the question, elements such as contextual information, examples of possible responses, a description of the desired structure and/or length of the output, or instructions on how the model should reason to derive the answer. This process of designing effective and complex prompts is referred to as prompt engineering.

Although prompt engineering plays a crucial role in controlling the model's behaviour, it often requires iterative fine-tuning and experimentation to achieve optimal results, as the same model may respond differently to similar prompts. As shown in [22], the selection of examples significantly affects the model's output during in-context learning. Similarly, the same prompt may produce varying results across different models. Prompt engineering can also help explore emergent properties in LLMs, particularly through in-context learning. This concept relates to a model's ability to perform tasks based solely on examples provided within the prompt, eliminating the need for further training or fine-tuning. Key types of in-context learning, as described by Brown et al. [21], include:

- **Zero-shot learning:** which occurs when the model understands a task solely from the description provided in the prompt and executes it without the need for examples.
- **One-shot learning:** which occurs when, in addition to the task description, only a single example is provided via the prompt for the model to understand the task.
- **Few-shot learning:** which occurs when, along with the task description, several examples are provided in the prompt for the model to understand and perform the requested task.

The Reasoning and Acting (ReAct) approach, introduced by Shinn et al. [23], extends prompt engineering by enabling dynamic interaction between models and their environment. It allows models to interact dynamically with their environment, supporting both decision-making and reasoning. For instance, a ReAct-based system in the cockpit could analyze weather data and suggest alternative routes in real-time while simultaneously verifying the feasibility of these routes by consulting air traffic constraints and fuel levels. This approach is particularly useful for tasks requiring complex decisions and the integration of information from multiple sources.

When analysing the use of ReAct to control an intelligent agent, it can be considered that this approach sits between classical AI planning [24], where an action plan is first generated to be subsequently executed, and the reactive approach proposed by Brooks [25], where an agent attempts to achieve a goal simply by interacting with the environment without creating a prior plan. In ReAct, a complex problem is decomposed into smaller issues described in the form of verbal reasoning by the LLM. Following this reasoning step, an action can be taken by interacting with the external world. This action will result in a change in the state of the world, and a description of this new state is received by the system in the form of an observation, which is then incorporated into the context of the world known to the model. The process repeats until the goal, which is the resolution of the problem, is achieved.

The ReAct approach operates in a sequence of reasonings, actions and observations in response to a question, as illustrated by the following example:

1. **Question:** *"What are the weather conditions at the destination airport?"*
2. **Reasoning:** *"To find out the weather conditions, I will consult the METAR."*
3. **Action:** Consult METAR.
4. **Observation:** To consult the METAR, a location needs to be provided.
5. **Reasoning:** *"To find out the destination airport, I need to consult the flight plan."*

Authorized licensed use limited to: ULAKBIM UASL - IZMIR YUKSEK TEKNOLOJI ENSTITUSU. Downloaded on February 01,2026 at 23:53:35 UTC from IEEE Xplore. Restrictions apply.

6. **Action:** Consult the flight plan.
7. **Observation:** Our destination is Malta International Airport.
8. **Reasoning:** *"Consult the METAR to determine the weather conditions at Malta International Airport."*
9. **Action:** Consult METAR.
10. **Observation:** The weather conditions at Malta International Airport are good: visibility above 10,000 metres and winds from the northwest at 1 knot.
11. **Response:** *"The weather conditions at our destination are good."*

To stimulate a similar sequence of reasoning, Shinn [26] proposes that, within the ReAct approach, a prompt template is employed which does not only outline the reasoning process following the cycle of Thought (reasoning), Action, and Observation but also includes four slots: {tools}, {tool_names}, {input}, and {agent_scratchpad}, as illustrated in Figure 2. During execution, the information corresponding to these slots is introduced as follows:

- **{tools}**: This slot contains the name, description, definition of input parameters, and return type of functions external to the model. These functions allow the model to interact with the real world, either by collecting information or modifying it. The functions that are part of the tool set can include simple calls to mathematical operations, internet queries, or more complex services, such as invoking other models.
- **{tool_names}**: This slot contains a simple list of the names of the tools (functions) that the model can use.
- **{input}**: This slot contains the user's question or questions created during the reasoning process.
- **{agent_scratchpad}**: During each execution cycle, the thoughts from the previous stages of the solution process, as well as intermediate results (observations), are placed in this slot.

Answer the following questions as best you can. You have access to the following tools:
{tools}
Use the following format:
Question: the input question you must answer
Thought: you should always think about what to do
Action: the action to take, should be one of {tool_names}
Action Input: the input to the action
Observation: the result of the action
... (this Thought/Action/Action Input/Observation can repeat N times)
Thought: I now know the final answer
Final Answer: the final answer to the original input question
Begin!
Question: {input}
Thought:{agent_scratchpad}

Figure 2. Template for a Reasoning and Acting prompt.

Retrieval-Augmented Generation

A limitation of LLMs is that they may generate outdated or incorrect answers when queried on information that can change or become stale beyond their training period [27]. This issue is particularly critical for tasks that require up-to-date and specific knowledge, such as operational procedures or regulations.

Retrieval-Augmented Generation (RAG), proposed by Lewis et al. [27], addresses this limitation by combining information retrieval with text generation. Instead of relying solely on stored knowledge, RAG retrieves relevant documents from a non-parametric database and incorporates this information into the model's response generation process. RAG has two main components: Retrieval and Generation.

A key technique used in RAG's retrieval process is the Dense Passage Retriever (DPR) [28], which encodes both documents and queries into dense vectors, a process known as embedding. By calculating the similarity between the query and document vectors, the system identifies the most relevant documents to incorporate into the LLM's generative process. DPR utilises Neural Networks (NNs) to generate dense representations for both queries and documents. This process can be divided into two key stages:

1. **Document Encoding**: All documents are encoded into dense vectors that semantically represent the content of each document. This process is carried out in a pre-processing phase and the documents are stored in a structure known as a Vector Database (or VectorDB), as shown in the left box of Figure 3.
2. **Query Encoding**: The query, such as a question, is encoded into a dense vector using an NN (typically transformer-based). After encoding both queries and documents into vectors, the system calculates the similarity between the query vector and the document vectors using metrics such as dot product. Documents with the highest similarity are considered the most relevant and are retrieved to be used as context in language generation tasks or other NLP tasks.

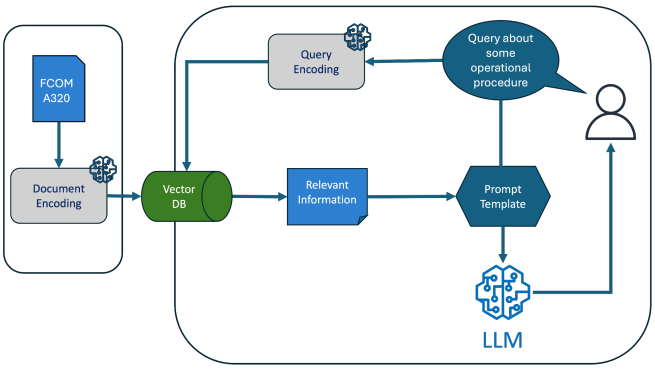


Figure 3. A potential system architecture based on the RAG approach, adapted from [24], designed to answer operational questions regarding the A320.

Authorized licensed use limited to: ULAKBIM UASL - IZMIR YUKSEK TEKNOLOJI ENSTITUSU. Downloaded on February 01,2026 at 23:53:35 UTC from IEEE Xplore. Restrictions apply.

In summary, RAG balances the generative capabilities of language models with the accuracy and timeliness provided by information retrieval, offering a powerful solution for tasks that require extensive and dynamic knowledge.

Fine-tuning

LLMs are often trained for generalisation, meaning they perform well across a broad range of tasks. However, when fine-tuned with niche data, the model can be specialised for specific tasks. This is useful in aviation applications such as, for example, adapting an LLM to understand terminology and acronyms used by pilots, or to interpret the meaning of a Terminal Aerodrome Forecast (TAF).

In essence, fine-tuning is a process that adapts pre-trained models to specific tasks using supervised learning. Typically, fine-tuning requires less data compared to the initial training, as the model already possesses prior knowledge. It is important to note that the amount of data required for fine-tuning depends on the task and the specificity of the adaptation. Some scenarios may require more data if the new task is complex or significantly different from the tasks the model was originally trained on.

One of the main challenges of fine-tuning LLMs is ensuring that the model does not lose the general understanding acquired during large-scale training [29]. This issue is known as catastrophic forgetting, where performance on previously learned tasks degrades when adapting to new ones. Additionally, as LLMs will normally have billions of parameters, fine-tuning all of them can be extremely costly in computational terms.

To address these challenges, Parameter-Efficient Fine-Tuning (PEFT) [30,31] techniques have been developed to adjust only a small set of parameters. PEFT includes various approaches, such as Adapters, Prompt Tuning, and Low-Rank Adaptation (LoRA), which modify only a fraction of the model's parameters. These techniques preserve the original weights of the LLM by introducing parameter adapters—smaller dimensional structures or additional layers that allow the model to adapt to new tasks by adjusting only a limited number of parameters without altering most of the original weights. As a result, PEFT also provides greater robustness against catastrophic forgetting, since the majority of the pre-trained weights remain unchanged.

In this context, LoRA is a commonly used method within PEFT. The principle behind LoRA is to reduce the number of trainable parameters during fine-tuning, thereby optimising GPU memory usage. This is achieved by freezing the pre-trained weights of the LLM and introducing low-rank matrices that re-parametrise the existing weights. Instead of modifying all parameters, LoRA focuses on adjusting the newly added low-rank matrices, which interact with the original model weights. This significantly reduces computational demands compared to full fine-tuning while preserving the model's intrinsic knowledge. LoRA is particularly beneficial for resource-constrained environments, enabling efficient fine-tuning without sacrificing performance in specialised tasks.

Language Agents

In the AI literature, multiple definitions of intelligent agents exist, ranging from philosophical to technical descriptions. This work adopts a technical perspective, aligned with the majority of definitions: an intelligent agent, or simply an agent, is a computational entity capable of perceiving its internal state and external environment, making decisions to achieve specific goals, and acting upon the world to achieve those goals [32-34]. Agents exhibit key characteristics such as autonomy, reactivity, proactivity and social ability — the capacity to interact and cooperate with other agents or humans [33-35].

Since 2021, solutions have emerged utilising agent-based AI architectures, in which an LLM serves as the control unit [36,37]. These systems are referred to as LLM-based agents or Language Agents. Unlike traditional AI agents, LLM-based agents possess a broader internal understanding of the world, enabling more informed decision-making without relying on domain-specific training. They leverage the common-sense knowledge embedded in LLMs to adapt to new tasks, minimising the need for human annotations or trial-and-error learning, which is common in traditional AI agents. Moreover, LLM-based agents can provide natural language interfaces for human interaction, making communication more flexible and explainable [36].

Pioneering systems that implement a Language Agent approach [38,39] propose a simple architecture in which the LLM operates within an event loop. In this loop, environmental observations are converted into text, which the LLM processes to decide the next action. The interaction between the LLM and the environment is facilitated by tools that translate the LLM's textual outputs into real-world actions and collect data from the environment. This data is then converted back into text and sent to the LLM, as illustrated in Figure 4.

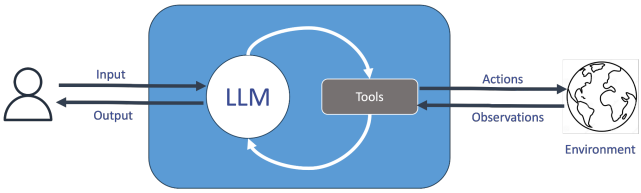


Figure 4. Representation of a Linguistic Agent.

A second generation of Language Agents has been proposed by various researchers [40,41], where agents are equipped with more complex internal structures, such as short-term and long-term memory [42], as well as fine-tuned prompts and LLMs to enhance planning [43] and decision-making capabilities [44]. These agents also have greater control over their internal states. These systems closely resemble cognitive agent models as proposed by [45] and are therefore referred to as Cognitive Language Agents (Figure 5).

Authorized licensed use limited to: ULAKBIM UASL - IZMIR YUKSEK TEKNOLOJI ENSTITUSU. Downloaded on February 01,2026 at 23:53:35 UTC from IEEE Xplore. Restrictions apply.

Projects such as Generative Agents [46], Voyager [47], and Reflexion [48] exemplify systems that follow this approach by incorporating advanced learning strategies, such as episodic memory, to store records of past interactions or events, akin to a 'journal' of the agent's experiences. This enables agents to identify patterns, such as which responses were effective and which errors occurred, thereby improving future interactions by avoiding mistakes and enhancing response quality.

The structure proposed by Shinn et al. [49] allows the agent to reflect on episodic memory to generate new semantic inferences. Similarly, the agents proposed by Wang et al. [50] use this reflection on episodic memory to modify their programming code, generating procedural knowledge and adapting their future behaviours based on past experiences.

In comparing Cognitive Language Agents with standalone LLMs, the key differences lie in the ability of Cognitive Language Agents to interact with the external world—a characteristic shared with Language Agents—and in the fact that Cognitive Language Agents are stateful, unlike the stateless nature of LLMs. Cognitive Language Agents use persistent memory structures to retain and update contextual information throughout interactions, allowing for dynamic decision-making and advanced reasoning. These agents can manage long-term dependencies in tasks and enhance pattern recognition, resulting in more personalised responses across a wide range of applications, from conversational interfaces to multi-step problem-solving.

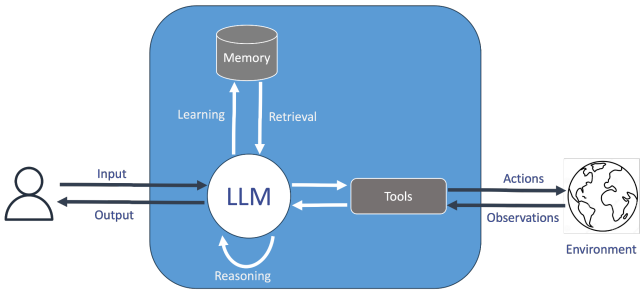


Figure 5. Representation of a Cognitive Linguistic Agent.

Multi-Agent Systems

A Multi-Agent System (MAS) consists of multiple autonomous agents that interact within an environment to achieve individual or collective objectives. These systems are particularly useful for complex tasks that can be distributed among different agents that either cooperate or compete [51].

According to the definition provided by Stone and Veloso [52], a MAS is "a loosely coupled network of problem-solving entities (agents) that work together to solve problems that are beyond the capabilities or knowledge of any single entity (agent)." While MAS designs are more complex than single-agent systems, they are well-suited for handling problems requiring diverse skills and knowledge, where constructing a single agent would be impractical. Multi-agent designs provide significant advantages in the context of Language Agents, namely:

- **Grouping of tools/responsibilities:** In an MAS, tasks can be assigned to specialised agents, increasing the likelihood of success by not overburdening a single agent with multiple tools.
- **Task-specific prompts:** Each agent can have prompts tailored to its specific task. When an LLM is provided with a prompt designed for a particular task, its performance improves considerably compared to when it is given a generic prompt. Research by [53] explores the benefits of personalised prompts for different tasks and the challenges posed by long and generic prompts.
- **LLMs fine-tuned for specific functions:** Each agent can use an LLM that is fine-tuned for its particular role within the system. This approach results in smaller fine-tuning datasets tailored to each agent's tasks, which reduces the risk of catastrophic forgetting. Furthermore, this method decreases the training time and allows for the use of smaller, dedicated models, making it more efficient compared to using a single model for all types of tasks.
- **Evaluation and optimisation:** The MAS architecture offers a conceptual framework that makes it easier to evaluate and improve individual agents without compromising the larger system.

One challenge in adopting an MAS approach is in determining the granularity the system should have. Larger, more generic agents may reduce specialisation of these agents, leading to sub-optimal task performance. Conversely, increasing the number of highly specialised agents may result in longer processing times and higher resource consumption. Additionally, continuous interaction and knowledge-sharing between agents increase the number of calls to the LLMs, leading to greater token usage and even higher computational demand.

3. Overview of the Proposed MA-LLM Architecture

Why a Multi-Agent Architecture?

LLMs are probabilistic models capable of generating multiple possible responses to a given query. A subset of these responses is generally more useful or relevant and is determined by factors such as the knowledge acquired during pre-training, fine-tuning for specific domains, and in-context learning via prompts, as illustrated in Figure 6.

In GenAI applications, balancing the base model's general knowledge, fine-tuning for task-specific adaptation and prompt engineering is critical. Achieving this balance becomes especially challenging in environments with strict operational constraints, where a set of valid responses is limited.

Authorized licensed use limited to: ULAKBIM UASL - IZMIR YUKSEK TEKNOLOJI ENSTITUSU. Downloaded on February 01,2026 at 23:53:35 UTC from IEEE Xplore. Restrictions apply.

A key consideration in defining a GenAI architecture for the flight deck is ensuring independent functionality, without reliance on externally hosted models. This allays safety, security and reliability concerns and also precludes the usage of large cloud-based models such as GPT-3.5 [21], GPT-4 [54], or Gemini [55] but then requires smaller, on-board models. Smaller models will obviously contain less parametric knowledge than the larger cloud-based counterparts and this will invariably increase the likelihood of generating inaccurate responses. Considering the trade-off between pre-trained knowledge, fine-tuning, and in-context learning, this limitation places greater demands on the latter two processes, which cannot fully compensate for the reduced knowledge. To mitigate this challenge, the proposed architecture divides complex tasks into simpler components using a multi-agent approach. Each agent specialises in distinct tasks, allowing the optimal base model to be selected. In some cases, this also reduces the need for extensive fine-tuning by providing task-specific knowledge through in-context learning.

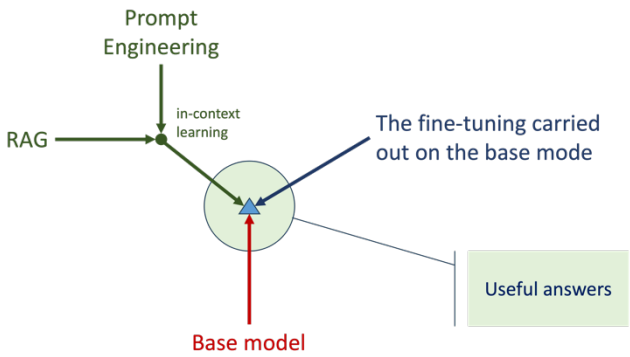


Figure 6. The response generated by a GenAI application is influenced by the model's parametric knowledge, the fine-tuning process, and the prompts.

For instance, a smaller model like Gemma2 [56], with approximately 9 billion parameters, can handle most aviation concepts and terminology, thus minimising the need for fine-tuning for most tasks. Hence, this architecture reduces latency by selecting smaller agents with efficient models for simple tasks, such as answering the question 'What is the IATA[1] code of Malta International Airport?'. More complex tasks, such as answering the question 'Is the wind speed at our destination airport greater than 20 knots?', require more resources and would therefore be assigned to more complex agents. Therefore, the multi-agent approach preserves computational resources by assigning simpler tasks to smaller agents. Additionally, it reduces the complexity of prompts, avoiding the need for large contextual inputs, which can slow down the response generation and complicates system debugging and maintenance.

One of the most critical advantages of the multi-agent architecture is its enhanced explainability and traceability of the generated solutions. LLMs are inherently black-box models, which makes explaining the reasoning behind suggested responses challenging. Unlike single-model architectures, where the reasoning behind responses is opaque, the multi-agent system breaks tasks into smaller components, and each agent's contribution can be logged and reviewed. This not only allows for the identification and debugging of failure points but also aids in understanding the problem-solving process employed by the system. This level of transparency is a key advantage of employing a multi-agent architecture and can ease the path to certification of AI systems on the flight deck.

System Architecture

The architecture combines Linguistic Agents and Cognitive Linguistic Agents, working collaboratively with the flight crew to resolve tasks in the cockpit. A Coordinator Agent synchronises tasks, ensuring no conflicts between agents and managing task dependencies. Each agent's internal architecture (LLMs, tools, storage structures, prompts, etc.) is designed based on the specific task it performs. The agents currently in use are categorised into three types, namely:

1. **Cognitive Agents**: These agents specialise in tasks that require complex inferential processes, like task decomposition. They are Cognitive Linguistic Agents and use prompts formatted in the ReAct style.
2. **Retriever Agents**: These agents are assigned tasks that require retrieving contextual information from different sources. They are responsible for querying vector databases to obtain specific data, such as information from documentation such as the Flight Crew Operating Manual (FCOM) or Aeronautical Information Publication (AIP), as necessary for the task at hand. By retrieving and combining the relevant information, these agents provide the required context to other agents. Retriever Agents employ different approaches to Retrieval-Augmented Generation, such as Naive RAG [28] and Adaptive-RAG [57].
3. **Wrapper Agents**: These agents interface the multi-agent system with external systems such as APIs providing meteorological data, avionics information, or other AI-supported systems. Wrapper Agents are used when the required context does not rely on parametric knowledge or vector databases but can be directly obtained from external sources. They utilise tools such as API calls, numerical optimisation methods and database queries, enabling the GenAI architecture to integrate efficiently with various data sources and provide real-time access to structured information.

Each agent's role and its interaction with the rest of the system are described in detail below and shown in Figure 7.

[1] **International Air Transport Association (IATA).**

Authorized licensed use limited to: ULAKBIM UASL - IZMIR YUKSEK TEKNOLOJI ENSTITUSU. Downloaded on February 01,2026 at 23:53:35 UTC from IEEE Xplore. Restrictions apply.

Coordinator Agent: This is the central coordinator and is responsible for task planning, breaking down complex tasks into smaller subtasks and assigning them to appropriate agents. With this approach, the concept of precedence is crucial, as the coordinator ensures that tasks are performed in the correct order and that each agent has all the necessary resources to fulfil the task. This agent falls under the Cognitive Agent category, where the prompt follows the ReAct pattern and contains information about the characteristics of each system agent, their specialties, and the data required for their correct operation. Additionally, a few-shot learning approach is used, in which examples of task decomposition and distribution among agents are incorporated into the prompt. To achieve better results in decomposing complex tasks, the LLM used in this agent was subjected to a fine-tuning process using a dataset containing complex tasks and their correct decomposition.

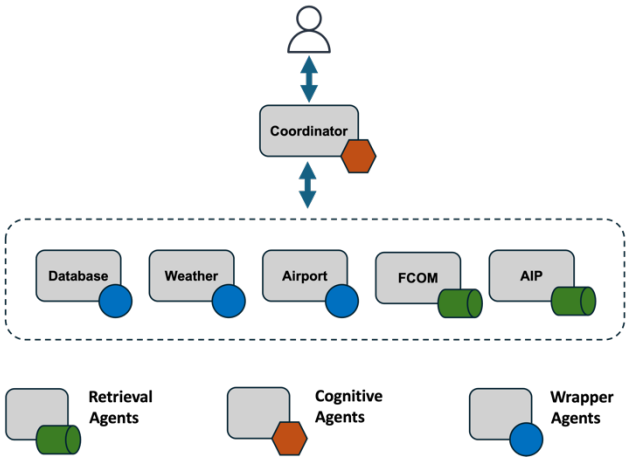


Figure 7. Current version of the proposed multi-agent GenAI system architecture.

Weather Agent: This is one of the Wrapper Agents, responsible for providing meteorological information. It uses an LLM to classify weather-related queries into two categories: weather reports or weather forecasts. The agent has a set of tools enabling it to query METAR servers for the current weather conditions at a particular location, and TAF servers for forecast weather data. Once the information is retrieved, the agent's LLM interprets the data and extracts the information necessary to answer the original query, which can vary widely (e.g., "What is the wind speed at LMML?" or "Is rain expected tomorrow at LFPG?"). The Gemma2 LLM is capable of interpreting METAR data via pre-training, but specific fine-tuning using the LoRA technique was applied for TAF interpretation. Supervised learning with synthetic data, generated using GPT-4, provided a robust training set to cover a wide range of meteorological scenarios. The agent's prompt includes examples of TAF interpretation to further enhance response accuracy.

Airport Agent: This Wrapper Agent is responsible for answering queries related to traffic conditions at a specific airport, such as "What are the current arrival delays at LMML?". The agent uses tools to access APIs that provide information on air traffic. This agent uses the Gemma2 LLM model, without the need for fine-tuning, as the queries are simple and the data received from the API is structured. These data are processed directly by the agent's tools and then sent to the LLM, which interprets them to generate a response to the query.

Database Agent: This Wrapper Agent accesses various databases containing information relevant to other agents, such as flight plans and tables with ICAO airport codes. When the agent receives a query, it determines the appropriate database to access. After retrieving the data, the internal LLM interprets the information and generates a response. This agent can handle queries such as "What is the ICAO code for Charles de Gaulle Airport?" or "What is the destination airport for this flight?". It uses the Gemma2 LLM model without any fine-tuning.

FCOM Agent: This is a Retriever Agent that contains a VectorDB with data from the aircraft's FCOM, enabling it to answer operational questions about the aircraft. In the current version, the agent uses the Adaptive-RAG technique, allowing it to dynamically retrieve and integrate relevant data. A value of k=3 is used, meaning that the agent retrieves three "chunks" of information related to the query's context, which are then combined by the LLM before providing a response to the requesting agent.

AIP Agent: This Retriever Agent contains a VectorDB with operationally important information about airports. In the current version, only the AIPs of the origin and destination airports are stored in the agent's VectorDB. Upon receiving a query, the LLM identifies the relevant database and combines the retrieved chunks to generate a contextualised response.

The two Retriever Agents log the pages of documentation from which chunks are extracted, and this information can be used to justify the response provided to the pilot, ensuring transparency and reliability. Both agents use the Gemma2 LLM model without any fine-tuning.

Selection of the Base Language Model

In the proposed architecture, the strategy is to standardize the use of a single type of LLM across all agents to ensure consistency and simplify integration. Although individual agents may eventually require fine-tuning to better align the LLM with their specific skills, the goal at this stage is to identify a robust base model that can serve all agents effectively. To achieve this, we adopted an experimental approach, comparing the performance of different LLMs on a shared task designed to evaluate their suitability.

The task chosen for this evaluation was decomposition of pilot queries. This task requires the LLM to demonstrate advanced reasoning capabilities by breaking down complex questions into simpler, well-defined sub-questions and assigning them to the appropriate agents. Such a task allows us to test not only the model's cognitive abilities (e.g.,

Authorized licensed use limited to: ULAKBIM UASL - IZMIR YUKSEK TEKNOLOJI ENSTITUSU. Downloaded on February 01,2026 at 23:53:35 UTC from IEEE Xplore. Restrictions apply.

reasoning, planning, and decision-making), but also its understanding of aviation-related terminology and concepts. These aspects are critical for evaluating the model's potential to operate effectively in a cockpit environment.

To select the base model for the agents, an initial dataset comprising 50 questions—typically asked by a pilot to the proposed system—was created. The questions were formulated according to the agents' areas of expertise and based on the terminology used by commercial pilots. The dataset includes both simple questions, which can be answered by a single agent, and complex questions, which need to be broken down into simpler components before being assigned to the appropriate agents.

Examples of complex questions that were included in the dataset are:

- Can an autoland be performed with a crosswind of 20 kts?
- The aircraft has transitioned to ALTN LAW. What are the possible reasons for this?
- What is the frequency of GOZO VOR/DME and is it currently in use?
- How will the developing weather system impact our planned route?

The task assigned to the language models in these experiments was twofold. First, the LLM needed to correctly identify which agent possessed the necessary skills to respond to a given question, based on the predefined relationship between agents and their respective expertise. Second, for complex questions, the LLM was expected to decompose the question into simpler, well-defined sub-questions. This decomposition process continued iteratively until every sub-question could be directly answered by one of the system's agents. A task was considered complete only if all sub-questions were correctly assigned to the relevant agents.

Various prompting strategies—Zero-shot, Few-shot, Chain of Thought (CoT), and ReAct—were tested on language models of different sizes, as shown in Table 1. GPT-3.5 and GPT-4 were included as benchmarks to establish baseline performance. Although these models were not directly implemented in the system, their results served as a reference for evaluating the effectiveness of the proposed models.

Figure 8 illustrates the number of tasks performed correctly by each language model. Throughout this work, the 'temperature' hyperparameter of the LLMs was set to 0 to ensure deterministic outputs, which is crucial for maintaining predictability in responses—an important factor for potential future system certification. Setting the temperature to 0 also minimizes the risk of hallucinations. Additionally, this configuration ensured that the outcome of each task executed by the same LLM remained consistent across multiple iterations, further reinforcing the reliability and repeatability of the results.

Table 1. Language models and parameters

Model	Number of parameters (billions)
Gemma2	9
GPT-3.5	175
GPT-4	1,760
LLAMA3	8
Phi3	2.2
Gemma2_27b	27

Figure 8 shows that GPT-4 performed best and this is expected, given the number of parameters it uses (Table 1). Of the non-benchmark language models, Figure 8 shows that Gemma2 performed best overall and results suggest that this model could meet the functional requirements of the agents within the system effectively. The Gemma2 model was therefore selected as the base language model.

4. Experimental Testing and Results

The primary objective of the experiments conducted in this stage of the project is to evaluate the feasibility of implementing a GenAI-based multi-agent architecture within the cockpit environment. The focus is to determine how effectively this architecture can handle pilot queries, manage task allocation, and provide accurate, contextualized responses without requiring a human-in-the-loop.

It is important to note that this evaluation does not yet address how the proposed architecture integrates with other aspects of the cockpit's HMI, such as touch panels, buttons, or other physical controls. Additionally, aspects involving human-in-the-loop considerations, such as pilot interactions with the system in real-time, have not yet been tested. These elements, including the use of simulators to assess the system's performance with different pilots under various scenarios, will be explored in future stages of the project.

By isolating these specific aspects, the current study aims to establish a baseline for the architecture's performance and reliability, focusing on critical components such as task decomposition, agent interaction, and response accuracy. This foundation will guide subsequent development and integration of the architecture into the broader cockpit environment, with a particular emphasis on pilot-system interaction and usability in realistic operational contexts.

Authorized licensed use limited to: ULAKBIM UASL - IZMIR YUKSEK TEKNOLOJI ENSTITUSU. Downloaded on February 01,2026 at 23:53:35 UTC from IEEE Xplore. Restrictions apply.

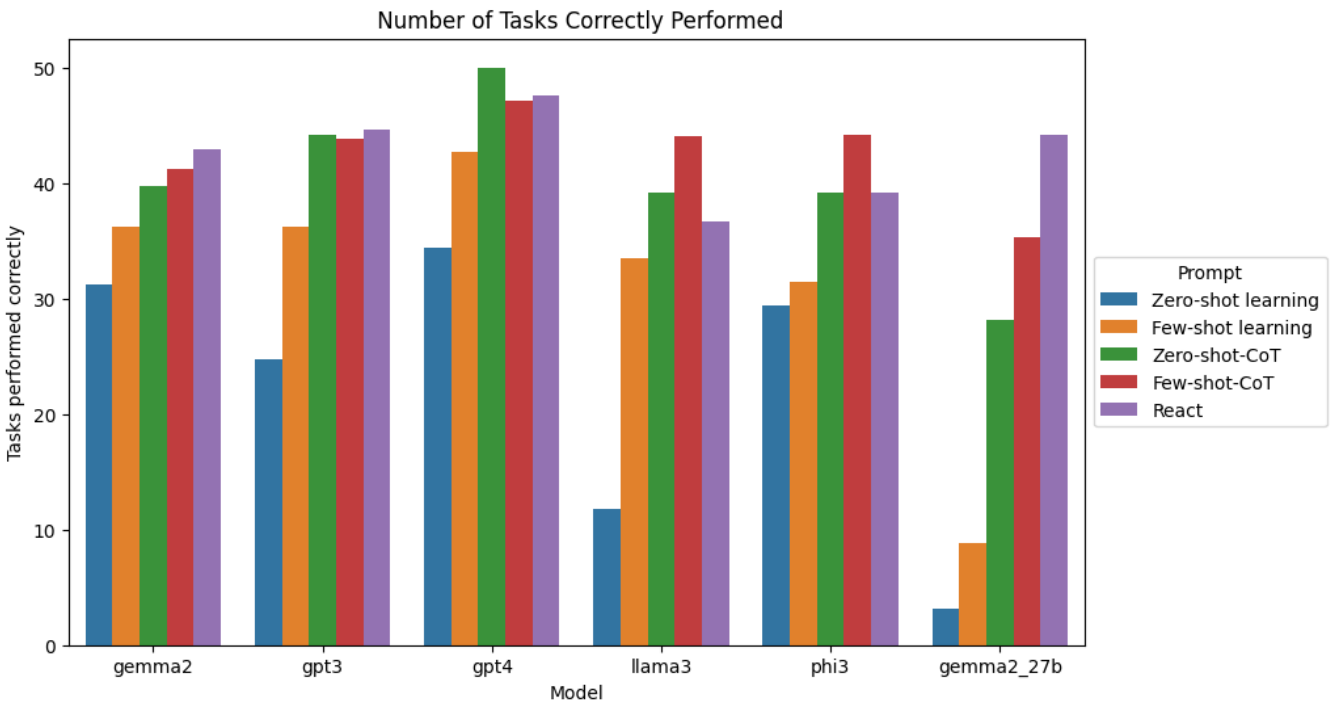


Figure 8. Comparison of the performance of different language models using various prompting approaches. Each model was tested for its ability to assign questions to the correct agents and, when necessary, to decompose complex questions into sub-questions and allocate them appropriately. The test set consists of 50 tasks and the graph shows the ones that performed correctly.

Test Dataset

An initial set of 16 questions that would typically be asked by a pilot was used to validate the system's architecture. This set was designed to test various aspects of the multi-agent system, covering topics that test its main functionalities while adhering to the agents' areas of expertise.

The initial focus was on testing the ability of the Coordinator Agent to decompose complex tasks and allocate them to the appropriate agents; and the ability of the selected agents to successfully perform the tasks assigned to them by the coordinator.

Table 2 presents the questions that were used, indicating whether they relate to static or dynamic data, as well as the agents involved in generating the responses. In addition to the agents listed in the table, the Coordinator Agent participates at least twice in the resolution process of each question - once to receive the question and once to provide the final answer.

Due to the non-deterministic nature of the system, the number of agents utilised could vary, depending on how the Coordinator Agent chose to break down the questions. For instance, the question "What is the current surface wind speed at the destination airport?" may be decomposed into two parts or sub-questions. A first sub-question could be "What is the destination airport?", a query which requires the Database Agent to retrieve the airport code. Once this is identified, the Weather Agent is queried with the second sub-question "What is the current surface wind speed at [destination airport code]?".

Alternatively, the Coordinator Agent may directly ask the Database Agent the question "What is the name of the destination airport?" and then separately request the ICAO code for that airport before finally querying the Weather Agent. This variability, while affecting agent utilisation, does not seem to compromise the accuracy of the generated answers.

To test the system's robustness in handling different formulations of the same request, a second set of questions was generated synthetically by GPT-4. For each of the 16 original questions in Table 2, 10 variations were created, altering the wording while retaining the original meaning. These variations made it possible to test the robustness of the system in handling different modes of expression as well as its sensitivity to minor alterations such as changes in word order, the use of synonyms, or more indirect reformulations, whilst retaining the same meaning of a question. For example, for the question 'What is the current surface wind speed at the destination airport?', variations such as 'Can you provide the current surface wind speed for the destination airport?' or 'What is the wind speed at the surface of the destination airport right now?' were generated. These reformulations help ensure that the system is still able to interpret correctly and respond appropriately to different forms of expression, maintaining accuracy and consistency in the responses.

Authorized licensed use limited to: ULAKBIM UASL - IZMIR YUKSEK TEKNOLOJI ENSTITUSU. Downloaded on February 01,2026 at 23:53:35 UTC from IEEE Xplore. Restrictions apply.

Table 2. Validation Questions

	Query	Data type	Agents used
1	Under which conditions is the COND TRIM AIR SYS FAULT alert triggered?	Static	FCOM
2	Can an autoland be performed with a crosswind of 20 kts?	Static	FCOM
3	Should a takeoff be continued if the aircraft speed exceeds V1?	Static	FCOM
4	What is the sequence of steps to be completed if there is an engine fire in flight?	Static	FCOM
5	What is the maximum weight (in 1000s of kg) for circling in CONF 3 with one engine inoperative when the OAT is 10 degrees Celsius and the airport elevation is 6000 feet?	Static	FCOM
6	What is the TORA of RWY 13 at Malta International Airport?	Static	AIP
7	What is the landing distance available for RWY 13?	Static	Database AIP
8	What are the runway lengths at the destination airport?	Static	Database AIP
9	What is the QNH at LMML?	Dynamic	Weather
10	What is the surface wind speed at LMML right now?	Dynamic	Weather
11	What is the current surface wind speed at the destination airport ?	Dynamic	Database Weather
12	What are the current arrival delays at LMML?	Dynamic	Airport
13	I would like to know the wind speed in km/h at our destination.	Dynamic	Database Weather
14	Is the wind at our destination airport greater than 30 knots?	Dynamic	Database Weather
15	What is the QNH in hPa at Frankfurt Airport?	Dynamic	Database Weather
16	Is the pressure at our destination airport less than 29.92 inHg?	Dynamic	Database Weather

These considerations yielded a dataset of a total of 176 questions, comprising the 16 original questions and their respective variations. The generation process was supervised to ensure that these variations maintained the original intent without introducing ambiguities that could affect the test.

General Results

The overall performance of the proposed architecture in answering the 176 questions is presented in Table 3. For comparison purposes, the architecture was evaluated using two different configurations, namely:

- Configuration 1: where GPT-4 was used as the primary LLM for all agents. GPT-4 was used as a reference to establish a comparative performance baseline.
- Configuration 2: where the Gemma2 model was used, with the Coordinator and Weather Agents fine-tuned for their specific tasks. For the other agents, a base version of Gemma2 was used without additional adjustments.

Table 3. Performance of the proposed architecture

Language Model	Number of Questions	Correct Answers	
GPT-4	176	157	89.2 %
Gemma2	176	146	82.9 %

To better understand these results, two separate analyses were carried out:

1. An evaluation of the Coordinator Agent's performance in decomposing complex tasks and allocating them to the appropriate agents. This evaluation considered key factors such as the accuracy of the task decomposition and the efficiency of agent allocation.

2. An evaluation of the other agents' ability to execute tasks that were correctly allocated, well-defined and provided with all necessary data for processing. This analysis emphasised the accuracy of the responses generated by the agents and the overall quality of task execution.

Authorized licensed use limited to: ULAKBIM UASL - IZMIR YUKSEK TEKNOLOJI ENSTITUSU. Downloaded on February 01,2026 at 23:53:35 UTC from IEEE Xplore. Restrictions apply.

The two analyses and their respective outcomes are described below.

Analysis of the Coordinator Agent

The Coordinator Agent was evaluated using GPT-4 as a benchmark for comparison. The Gemma2 model was also evaluated under two conditions: without task-specific fine-tuning and with fine-tuning through supervised learning. The fine-tuning was aimed at improving Gemma2's ability to efficiently decompose complex tasks and assign each sub-task to the appropriate agent. The training involved a diverse dataset of complex tasks and their corresponding correct decompositions, ensuring variety and quality in the learning process. This approach allowed for a comparison of Gemma2's performance before and after supervised training and the evaluation of the effectiveness of the two conditions relative to the reference model under different conditions.

Table 4 presents the recorded performance of the Coordinator Agent using the GPT-4 and Gemma2, the latter with the described two conditions, when tested on the 16 questions (Table 2) and on the extended dataset of 176 questions. GPT-4 demonstrated a high accuracy in both test sets (that is, when using both the 16 questions and the extended dataset of 176 questions). This can be attributed not only to GPT-4's advanced comprehension abilities but also to the fact that it was used to generate a significant portion of the 176 query dataset. This can be expected to introduce some level of bias in the result, as the model will have had prior familiarity with the question patterns, thereby increasing the probability that the question will be decomposed correctly or as desired by the agent.

Table 4. Performance of the Coordinator Agent

Language Model	Number of Questions	Correct Decomposition & Agent Allocation	
GPT-4	16	16	100 %
Gemma2 without fine-tuning	16	16	100 %
Gemma2 with fine-tuning	16	16	100 %
GPT-4	176	174	97.4 %
Gemma2 without fine-tuning	176	141	80.1 %
Gemma2 with fine-tuning	176	162	92.0%

Gemma2 performed just as well (correctly decomposing the query and allocating agents) on all 16 questions of the original set (Table 2). This can be attributed to the unintended influence of the initial question set on the agent's prompt design, which inadvertently specialised the agents to handle the specific patterns and structures present in those queries. However, this same bias that enhanced performance on the original set also led to reduced effectiveness when the system was tested on the extended dataset of 176 questions. The broader variations in question formulation in the extended set exposed limitations in the agents' ability to generalise beyond the specific patterns they had become specialised in handling. Fine-tuning through supervised learning significantly improved the model's ability to generalise and handle varied questions. Table 4 shows an improvement of approximately 12% (162 correct answers compared to 141 without fine-tuning), demonstrating a greater capacity for adaptation to different forms of expression when the model is fine-tuned.

The impact of fine-tuning becomes clearer when examining the results across two types of questions: simple questions, which do not require task decomposition, and complex questions, which must be broken down into sub-tasks. Simple questions are those that can be answered directly by a single agent, such as "What is the ICAO code of the destination airport?". In contrast, complex questions require multiple steps, such as "What is the current surface wind speed at the destination airport?", which first involves determining the destination airport before retrieving meteorological information.

To better illustrate how complex questions are handled by the system, Figure 9 provides an example. It illustrates how a complex question, such as "Is the wind at our destination airport greater than 30 knots?", is decomposed into simpler queries and subsequently sent to the relevant agents. The figure shows the thought processes of each agent and the exchange of messages between them during the operation. Finally, the Coordinator Agent consolidates the responses and provides the final answer to the pilot.

Table 5 provides a comparison of the system's performance on a set of 176 questions, with 99 classified as simple and the rest as complex. The classification was based on the number of steps required and the need for interaction among multiple agents to answer each question.

Table 5. Performance of the Coordinator Agent in handling simple and complex questions

Language Model	Correct Decomposition & Agent Allocation	
99 Simple Questions		
GPT-4	99	100 %
Gemma2 without fine-tuning	97	97.9 %
Gemma2 with fine-tuning	99	100 %
77 Complex Questions		
GPT-4	75	97.4 %
Gemma2 without fine-tuning	44	57.1 %
Gemma2 with fine-tuning	63	81.0%

Authorized licensed use limited to: ULAKBIM UASL - IZMIR YUKSEK TEKNOLOJI ENSTITUSU. Downloaded on February 01,2026 at 23:53:35 UTC from IEEE Xplore. Restrictions apply.

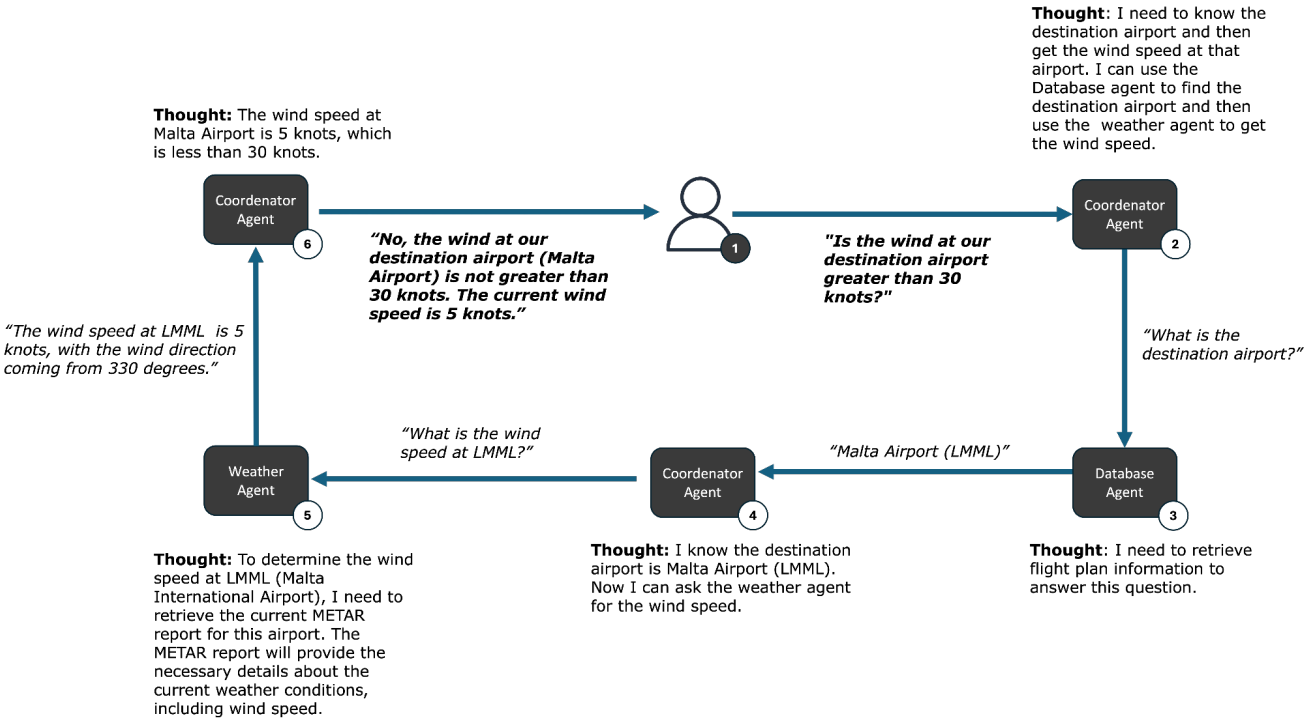


Figure 9. Example of architecture functionality—resolution process of Question 14 from Table 2, illustrating the thoughts, questions, and responses of the agents as extracted from system logs.

From Table 5 it can be observed that, after fine-tuning, the performance of Gemma2 on complex questions improved by 24%. This result demonstrates the model's enhanced capacity for generalisation, indicating that fine-tuning was effective in improving Gemma2's ability to handle multiple-step tasks.

Although the current stage of the project does not establish specific minimum response time requirements for the system to answer pilot queries, the importance of processing time for the project's ultimate feasibility cannot be understated. To better understand and optimise this aspect, the processing time was divided into three distinct phases: the time taken by the Coordinator Agent to decompose complex tasks and allocate them to other agents, the time required for the execution of these tasks by the respective agents, and the time spent by the Coordinator Agent in aggregating the responses from these agents to generate the final answer. Given the hardware limitations and the sequential execution of agents in this phase of the project, measuring the total time from query to final response would not accurately reflect the potential efficiency of the multi-agent architecture. Therefore, this evaluation focused exclusively on the Coordinator Agent's processing time, as it represents a central and iterative element of the task resolution process, as shown in Figure 9.

The results indicate that the average processing time for the Coordinator Agent to respond to simple queries was 11 seconds while, for complex queries, it increased to 37 seconds. Although these absolute times can be reduced with improved computational hardware, the data suggest a tendency for the increase in processing time to not scale linearly with the complexity of the tasks. This observed trend underscores the importance of further investigating and optimising the architecture to efficiently handle more complex queries. Future iterations should also aim to leverage the parallelism inherent in the multi-agent approach, which is expected to significantly reduce overall response times.

Analysis of the Other Agents

When analysing the performance of the other agents, it is important to consider that task distribution among agents is not uniform. This disparity arises due to the specialisation of each agent for different types of tasks. For instance, in the set of 16 questions described in Table 1, the Weather Agent was used to answer 7 questions, while the Airport Agent was used only once. Furthermore, only tasks that were correctly decomposed and allocated to the appropriate agent(s) were included in the analysis. This filtering was performed by examining the system log files, ensuring that only cases where the Coordinator Agent correctly executed the decomposition and allocation were included in the analysis.

The difference in the nature of tasks performed by the agents led to varying performance levels, regardless of the agent's main LLM (i.e. GPT-4 or Gemma2):

- **Wrapper Agents** (such as Weather, Airport, and Database agents) had a success rate close to 100% for all requests. These agents act primarily as intermediaries, packaging and forwarding information, often interacting directly with external APIs or databases. Their high success rates are expected due to the straightforward nature of their

Authorized licensed use limited to: ULAKBIM UASL - IZMIR YUKSEK TEKNOLOJI ENSTITUSU. Downloaded on February 01,2026 at 23:53:35 UTC from IEEE Xplore. Restrictions apply.

task, with all the necessary context provided by the Coordinator Agent.

- **RAG Agents** (such as FCOM and AIP agents) showed lower success rates, around 80%. Despite the use of efficient RAG approaches, the lower performance can be attributed to the format of the documents populating their vector databases. Documents such as the FCOM and AIP contain numerous images and tables, which complicate the embedding process.

The embedding process converts text (and other types of information) into numerical representations, or vectors, which can be understood by language models. However, the presence of images and tables complicates this conversion, as these structures contain non-textual information, much of which is specific to the aviation domain, such as visualising procedures for approaching and landing on a particular runway. This information cannot be easily converted into meaningful vectors without advanced preprocessing. This limitation reduces the accuracy of the embeddings and, consequently, the effectiveness of the agents in retrieving and generating relevant responses. The results indicate that effective handling of these structures—such as advanced techniques for dealing with images and tables—should be considered in future versions of the architecture to enhance the effectiveness of RAG agents.

This comparison aims to demonstrate the efficiency, reliability, and suitability of the proposed system for cockpit use, ensuring high availability and fault tolerance without reliance on external cloud infrastructure.

5. Discussion and Conclusion

The experimental evaluation of the proposed GenAI-based multi-agent architecture demonstrates its potential to support decision-making in the flight deck through the efficient allocation of specialised tasks to agents. The results indicate that the architecture, which integrates Adaptive-RAG and smaller specialised models such as Gemma2, achieves a high degree of accuracy in task decomposition, agent assignment, and information retrieval, with the potential to effectively reduce pilot cognitive workload.

The evaluation was carried out using two configurations: one where GPT-4 served as the primary language model for all agents, and another where the Gemma2 model was used with specific fine-tuning for some agents. The GPT-4 model exhibited strong performance across both simple and complex tasks, due to its extensive parametric knowledge and advanced natural language processing capabilities. However, the fine-tuned Gemma2 model also demonstrated robust performance, particularly after adaptation, suggesting that smaller, domain-optimised models can achieve comparable results when appropriately tuned.

The results for the Coordinator Agent showed that this agent could correctly decompose complex tasks and assign them to appropriate agents. However, the success rate for handling complex tasks was 81%, providing a significant opportunity for further improvement in managing more intricate tasks.

Wrapper Agents such as the Weather and Database Agents achieved near-perfect success rates, reflecting their efficiency in handling structured data and straightforward queries. However, the RAG Agents, which are tasked with retrieving and interpreting complex documents, showed lower performance at around 80%, irrespective of the underlying LLM used. This lower level of effectiveness is likely due to the nature of the documents they process, where the process of embedding non-textual information poses challenges for the LLMs. Addressing this limitation through advanced embedding techniques is essential for enhancing the performance of RAG agents in future iterations.

It is relevant to emphasise that, due to the operational requirements of the commercial flight deck, deploying a model like GPT-4 onboard is not currently feasible due to its size and reliance on external cloud infrastructure. In this work GPT-4 was used solely for benchmarking purposes, providing a reference to evaluate the performance of other, more practical models like Gemma2.

The Coordinator Agent plays a key role in this architecture, and any malfunction could compromise the entire system. To mitigate this risk, two main directions for future work are suggested, namely: improving the efficiency of the Coordinator Agent in handling complex tasks and distributing its functions across other agents in the system to reduce the risk of a single point of failure.

Furthermore, due to the safety-critical nature of flight deck operations, the architecture should include agents focused on the explainability of the results. This would provide transparency in the problem-solving process, enhancing pilot trust and facilitating certification of the system.

The current stage of the project does not fully address certain critical aspects of human-machine interaction and human-in-the-loop considerations. For example, while the time required to generate responses was analysed to some extent, a more in-depth evaluation is needed to assess its impact on timely and effective decision-making. Additionally, the modality in which these responses are presented to the pilot—another key factor for operational efficiency—was not a focus of this phase. All responses in the current system were presented in text format, and a detailed measurement of response times was not undertaken. These limitations underscore the need for a more comprehensive exploration of HMI-related aspects in the next phase of the project.

Future efforts will focus on addressing current limitations and advancing the architecture to prepare for initial testing with pilots in a simulator environment. Key priorities include improving system resilience, enhancing explainability, and addressing critical HMI challenges to ensure the architecture aligns with practical and regulatory requirements in the aviation industry.

Authorized licensed use limited to: ULAKBIM UASL - IZMIR YUKSEK TEKNOLOJI ENSTITUSU. Downloaded on February 01,2026 at 23:53:35 UTC from IEEE Xplore. Restrictions apply.

Specific areas of development will include:

- Tackling the challenge of embedding non-textual information, such as images and tables, to enhance the performance of RAG agents when processing complex aviation documents.
- Optimising model fine-tuning and improving prompt engineering to increase the efficiency and accuracy of agents in handling both simple and complex tasks.
- Exploring architectural modifications to distribute the responsibilities of the Coordinator Agent, thereby mitigating the risk of a single point of failure and increasing system robustness.
- Incorporating explainability-focused agents to provide transparency in the problem-solving process, fostering pilot trust and aiding in system certification.
- Evaluating response generation times to ensure real-time compatibility with flight deck operations and exploring different response modalities, such as visual, auditory, or multimodal formats, to determine the most effective way to present information to pilots.

The ultimate goal is to integrate these improvements and conduct human-in-the-loop experiments in flight simulators. These tests will assess how the system influences pilot performance and workload in realistic scenarios, paving the way for further refinements and eventual deployment in operational environments.

ACKNOWLEDGEMENTS

The work described in this paper was carried out as part of the ArtiAP (R&I-2021-014-T) project which is financed by the Malta Council for Science & Technology, for and on behalf of the Foundation for Science and Technology, through the FUSION: R&I Technology Development Programme.

REFERENCES

[1] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong, et al., "A survey of large language models," arXiv preprint, arXiv:2303.18223, 2023. [Online]. Available: https://arxiv.org/abs/2303.18223.

[2] C. Yang and C. Huang, "Natural Language Processing (NLP) in Aviation Safety: Systematic Review of Research and Outlook into the Future," Aerospace, vol. 10, no. 7, p. 600, 2023. doi: 10.3390/aerospace10070600.

[3] L. Wang, J. Chou, X. Zhou, A. Tien, and D. M. Baumgartner, "Aviationgpt: A large language model for the aviation domain," 2023.

[4] A. T. Ray, B. F. Cole, O. J. P. Fischer, R. T. White, and D. N. Mavris, "aeroBERT-Classifier: Classification of Aerospace Requirements Using BERT," Aerospace, vol. 10, p. 279, 2023. doi: 10.3390/aerospace10030279.

[5] Y. Liu, "Large language models for air transportation: A critical review," Journal of the Air Transport Research Society, vol. 2, June 2024. doi: 10.1016/j.jatrs.2024.100024.

[6] J. Andriuškevičius and J. Sun, "Automatic Control With Human-Like Reasoning: Exploring Language Model Embodied Air Traffic Agents," *arXiv preprint arXiv:2409.09717*, 2024. Available: https://arxiv.org/abs/2409.09717.

[7] European Union Aviation Safety Agency, "ARTIFICIAL INTELLIGENCE ROADMAP 2.0: Human-centric approach to AI in aviation," Sep. 2020. [Online]. Available: https://www.easa.europa.eu/en/document-library/general-publications/easa-artificial-intelligence-roadmap-20. [Accessed: 01-Oct-2024].

[8] EASA, 'Concept Paper: Guidance for Level 1 & 2 machine learning applications — Proposed Issue 02', European Union Aviation Safety Agency (EASA), Cologne, 2023.

[9] T. Mikolov, M. Karafiát, L. Burget, J. Černocký, and S. Khudanpur, "Recurrent neural network based language model," in INTERSPEECH 2010, 11th Annual Conference of the International Speech Communication Association, Makuhari, Chiba, Japan, Sep. 26-30, 2010, T. Kobayashi, K. Hirose, and S. Nakamura, Eds., ISCA, 2010, pp. 1045–1048.

[10] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," Neural Computation, vol. 9, no. 8, pp. 1735-1780, 1997. doi: 10.1162/neco.1997.9.8.1735.

[11] K. Cho, B. van Merriënboer, D. Bahdanau, and Y. Bengio, "On the Properties of Neural Machine Translation: Encoder-Decoder Approaches," arXiv preprint, arXiv:1409.1259, 2014. [Online]. Available: https://arxiv.org/abs/1409.1259.

[12] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, Dec. 4-9, 2017, Long Beach, CA, USA, pp. 5998-6008.

[13] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: pre-training of deep bidirectional transformers for language understanding," in Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, Jun. 2-7, 2019, vol. 1, J.

Authorized licensed use limited to: ULAKBIM UASL - IZMIR YUKSEK TEKNOLOJI ENSTITUSU. Downloaded on February 01,2026 at 23:53:35 UTC from IEEE Xplore. Restrictions apply.

Burstein, C. Doran, and T. Solorio, Eds., Association for Computational Linguistics, 2019, pp. 4171-4186.

[14] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, "Improving language understanding by generative pre-training," OpenAI, 2018. [Online]. Available: https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf

[15] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language Models are Unsupervised Multitask Learners," OpenAI, 2019. [Online]. Available: https://www.techbooky.com/wp-content/uploads/2019/02/Better-Language-Models-and-Their-Implications.pdf.

[16] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer, "BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension," in Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, Jul. 2020, pp. 7871-7880. doi: 10.18653/v1/2020.acl-main.703.

[17] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling laws for neural language models," arXiv preprint, arXiv:2001.08361, 2020. [Online]. Available: https://arxiv.org/abs/2001.08361.

[18] J. Wei, Y. Tay, R. Bommasani, C. Raffel, B. Zoph, S. Borgeaud, D. Yogatama, M. Bosma, D. Zhou, D. Metzler, E. H. Chi, T. Hashimoto, O. Vinyals, P. Liang, J. Dean, and W. Fedus, "Emergent abilities of large language models," CoRR, vol. abs/2206.07682, 2022. [Online]. Available: https://arxiv.org/abs/2206.07682.

[19] Z. Du, A. Zeng, Y. Dong, and J. Tang, "Understanding Emergent Abilities of Language Models from the Loss Perspective," arXiv, 2024. [Online]. Available: https://arxiv.org/abs/2403.15796.

[20] T. Schick and H. Schütze, "It's not just size that matters: Small language models are also few-shot learners," in Proceedings of NAACL, 2021, pp. 2339-2352. [Online]. Available: https://aclanthology.org/2021.naacl-main.185.

[21] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," in Advances in Neural Information Processing Systems, vol. 33, Curran Associates, Inc., 2020, pp. 1877-1901.

[22] J. Liu, D. Shen, Y. Zhang, B. Dolan, L. Carin, and W. Chen, "What makes good in-context examples for GPT-3?," in Proceedings of Deep Learning Inside Out (DeeLIO 2022): The 3rd Workshop on Knowledge Extraction and Integration for Deep Learning Architectures, Dublin, Ireland and Online, Association for Computational Linguistics, 2022, pp. 100-114.

[23] A. Shinn, E. Labash, Y. Li, Y. Wu, and N. D. Goodman, "ReAct: Synergizing Reasoning and Acting in Language Models," arXiv preprint, arXiv:2210.03629, 2023. [Online]. Available: https://arxiv.org/abs/2210.03629.

[24] M. Ghallab, D. Nau, and P. Traverso, Automated Planning: Theory and Practice, San Francisco, CA, USA: Morgan Kaufmann, 2004.

[25] R. A. Brooks, "Intelligence without reason," in Proceedings of the 12th International Joint Conference on Artificial Intelligence - Volume 1, Morgan Kaufmann Publishers Inc., 1991, pp. 569-595.

[26] A. Shinn, E. Labash, Y. Li, Y. Wu, and N. D. Goodman, "ReAct: Synergizing Reasoning and Acting in Language Models," arXiv preprint, arXiv:2210.03629, 2023. [Online]. Available: https://arxiv.org/abs/2210.03629.

[27] S. M. Mousavi, S. Alghisi, and G. Riccardi, "DyKnow: Dynamically Verifying Time-Sensitive Factual Knowledge in LLMs," arXiv preprint arXiv:2404.08700, 2024.

[28] K. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Proceedings of Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 9459-9474.

[29] Y. Luo, Z. Yang, F. Meng, Y. Li, J. Zhou, and Y. Zhang, "An Empirical Study of Catastrophic Forgetting in Large Language Models During Continual Fine-tuning," arXiv preprint arXiv:2308.08747v3, 2024.

[30] Z. Lin, A. Madotto, and P. Fung, "Exploring versatile generative language model via parameter-efficient transfer learning," in Findings of the Association for Computational Linguistics: EMNLP 2020, Association for Computational Linguistics, Online, 2020, pp. 441-459.

[31] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," arXiv preprint,

Authorized licensed use limited to: ULAKBIM UASL - IZMIR YUKSEK TEKNOLOJI ENSTITUSU. Downloaded on February 01,2026 at 23:53:35 UTC from IEEE Xplore. Restrictions apply.

arXiv:2106.09685, 2021. [Online]. Available: https://arxiv.org/abs/2106.09685.

[32] G. E. M. Anscombe, Intention, Harvard University Press, 2000.

[33] S. J. Russell, Artificial Intelligence: A Modern Approach, Pearson Education, Inc., 2010.

[34] M. J. Wooldridge and N. R. Jennings, "Intelligent agents: theory and practice," Knowledge Engineering Review, vol. 10, no. 2, pp. 115–152, 1995.

[35] R. Goodwin, "Formalizing properties of agents," Journal of Logic and Computation, vol. 5, no. 6, pp. 763–781, 1995.

[36] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z.-Y. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J.-R. Wen, "A survey on large language model based autonomous agents," arXiv, arXiv:2308.11432, 2023. [Online]. Available: https://arxiv.org/abs/2308.11432.

[37] T. R. Sumers, S. Yao, K. Narasimhan, and T. L. Griffiths, "Cognitive architectures for language agents," arXiv preprint, arXiv:2309.02427, 2024. [Online]. Available: https://arxiv.org/abs/2309.02427.

[38] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, et al., "Do as I can, not as I say: Grounding language in robotic affordances," arXiv preprint, arXiv:2204.01691, 2022. [Online]. Available: https://arxiv.org/abs/2204.01691.

[39] W. Huang, P. Abbeel, D. Pathak, and I. Mordatch, "Language models as zero-shot planners: Extracting actionable knowledge for embodied agents," in International Conference on Machine Learning, 2022, pp. 9118-9147.

[40] R. E. Wray, J. R. Kirk, J. E. Laird, et al., "Language models as a knowledge source for cognitive agents," arXiv preprint, arXiv:2109.08270, 2021. [Online]. Available: https://arxiv.org/abs/2109.08270.

[41] O. J. Romero, J. Zimmerman, A. Steinfeld, and A. Tomasic, "Synergistic integration of large language models and cognitive architectures for robust AI: An exploratory analysis," arXiv preprint, arXiv:2308.09830, 2023. [Online]. Available: https://arxiv.org/abs/2308.09830.

[42] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," arXiv preprint, arXiv:2304.03442, 2023. [Online]. Available: https://arxiv.org/abs/2304.03442.

[43] S. Hao, Y. Gu, H. Ma, J. J. Hong, Z. Wang, D. Z. Wang, and Z. Hu, "Reasoning with language model is planning with world model," arXiv preprint, arXiv:2305.14992, 2023. [Online]. Available: https://arxiv.org/abs/2305.14992.

[44] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," arXiv preprint, arXiv:2210.03629, 2022. [Online]. Available: https://arxiv.org/abs/2210.03629.

[45] P. Langley, J. E. Laird, and S. Rogers, "Cognitive architectures: Research issues and challenges," Cognitive Systems Research, vol. 10, no. 2, pp. 141-160, 2009.

[46] E. Park, J. Liang, and S. C. Oh, "Generative Agents: Interactive Simulacra of Human Behavior," arXiv preprint arXiv:2304.03442, 2023.

[47] G. Wang, H. Gao, and S. Oh, "Voyager: An Open-Ended Embodied Agent with Large Language Models," arXiv preprint arXiv:2305.16291, 2023.

[48] X. Hou, T. Jiang, and X. Zhou, "Reflexion: Language Agents with Adaptive Feedback Memory," arXiv preprint arXiv:2303.11366, 2023.

[49] N. Shinn, F. Cassano, B. Labash, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," arXiv preprint, arXiv:2303.11366, 2023. [Online]. Available: https://arxiv.org/abs/2303.11366.

[50] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," arXiv preprint, arXiv:2305.16291, 2023. [Online]. Available: https://arxiv.org/abs/2305.16291.

[51] J. Rocha, Ed., Multi-agent Systems. InTech, Sep. 13, 2017. doi: 10.5772/66595.

[52] P. Stone and M. Veloso, "Multiagent Systems: A Survey from a Machine Learning Perspective," Autonomous Robots, vol. 8, pp. 345–383, 2000. doi: https://doi.org/10.1023/A:10089420122

[53] K. Chang, S. Xu, C. Wang, Y. Luo, T. Xiao, and J. Zhu, "Efficient prompting methods for large language

Authorized licensed use limited to: ULAKBIM UASL - IZMIR YUKSEK TEKNOLOJI ENSTITUSU. Downloaded on February 01,2026 at 23:53:35 UTC from IEEE Xplore. Restrictions apply.

models: A survey," CoRR, vol. abs/2404.01077, 2024. [Online]. Available: https://arxiv.org/abs/2404.01077.

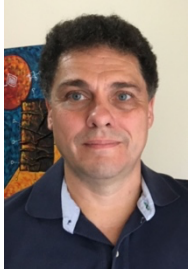
[54] OpenAI, "GPT-4 Technical Report," arXiv preprint, arXiv:2303.08774, 2023. [Online]. Available: https://arxiv.org/abs/2303.08774.

[55] Gemini Team, R. Anil, S. Borgeaud, J.-B. Alayrac, J. Yu, R. Soricut, J. Schalkwyk, A. M. Dai, A. Hauth, K. Millican, et al., "Gemini: A Family of Highly Capable Multimodal Models," arXiv preprint, arXiv:2312.11805, 2024. [Online]. Available: https://arxiv.org/abs/2312.11805.

[56] C. Farabet and T. Warkentin, "Gemma 2 is now available to researchers and developers," Google Blog, 2024. [Online]. Available: https://blog.google/technology/developers/google-gemma-2/.

[57] S. Jeong, J. Baek, S. Cho, S. J. Hwang, and J. C. Park, "Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity," arXiv preprint, arXiv:2403.14403, 2024. [Online]. Available: https://arxiv.org/abs/2403.14403.

BIOGRAPHY



José Prado *received his B.Sc. in Computer Science from the Federal University of São Carlos (1988), his M.Sc. in Computer Science and Computational Mathematics from ICMSC-USP (1991), and his Ph.D. in Electrical Engineering from POLI-USP (1996). He coordinated the CNPq SET-Level A Project on Innovation and Technological Development in Robotics, Autism Spectrum, and Language Development. He has worked on various technology projects, authored educational books on robotics and programming, and holds patents. Currently, he focuses exclusively on AI applications in aerospace.*

David Zammit-Mangion *is the Director of the Institute of Aerospace Technologies at the University of Malta. He is an industry-oriented academic, lecturing at the University of Malta and Cranfield University (UK). His professional focus is on industrial research and international collaboration; technology transfer to industry; and graduate and post-graduate education with industrial application. His principal research areas are: avionics surveillance systems, cockpit alerting, cockpit human machine interface and interaction (HMI2) design and evaluation; air traffic management; guidance, navigation and control applications, trajectory and mission optimisation. He is the author of over 100 academic publications and 100 research documents and is the holder of several patents.*

Brian Zammit *has been active in academic research since joining the University in 2005. He earned his PhD from the University of Malta in 2015 and is now a senior lecturer in the Department of Electronic Systems Engineering at the University of Malta. His research focuses on aircraft operations, optimisation, and artificial intelligence applications for air traffic management and on-board crew support.*

Jason Gauci *received his B.Elec.Eng.(Hons) degree in engineering from the University of Malta in 2005 and his PhD in aerospace from Cranfield University in 2010. He is currently a senior lecturer at the Institute of Aerospace Technologies at the University of Malta and an adjunct assistant professor at Embry-Riddle Aeronautical University (Worldwide). His research interests include: avionics, air traffic management, unmanned systems, and machine learning for aviation applications. He has been - and is currently involved - in several national and EU-funded research projects and is the holder of a number of patents.*

Alan Muscat *is a former Airline Captain with many years of commercial flight experience on Airbus and Boeing aircraft, as well as other types. He received his B.Sc. (Hons) degree in Computing and Information Systems from the University of London, Goldsmiths College in 2007. He is one of the founders and currently sole Director of QuAero. In recent years he has been involved in projects and with companies who are working on the development and use of Unmanned Aircraft Systems. He originally got involved in the research sector in 2005 when he was involved in an EU-funded research project. Since 2010, through QuAero, he has actively participated in many nationally funded research projects. He also holds several patents in his name.*

Authorized licensed use limited to: ULAKBIM UASL - IZMIR YUKSEK TEKNOLOJI ENSTITUSU. Downloaded on February 01,2026 at 23:53:35 UTC from IEEE Xplore. Restrictions apply.

Sandro Mizzi *is an A320 pilot with nearly 11,000 flying hours and work experience in various fields in the aviation industry. Sandro's career encompasses multiple roles, including positions at the meteorological office and the aviation regulator, where he served as part of the Flight Operations Inspectorate. He has also worked as an aviation manager in a into plane fuel operation at Malta International Airport. He earned his BSc (Hons) in Computing and Information Technology from Derby University and is pursuing a Master's in Ergonomics and Human Factors. Currently, Sandro is the Deputy Director of Flight Operations at Dronamics, focusing on innovative solutions in the aviation field.*

Andre Manicol*o received his Bachelor's (Hons.) degree in Electrical Engineering from the University of Malta. He is an airline pilot with over 12 years of operational experience and more than 8,000 flying hours on the Airbus A320. He is currently flying as a senior first officer with KM Malta Airlines. In recent years, he was conducting research with the Institute of Aerospace Technology at the University of Malta. The primary focus of his research was to explore the potential for recovering energy from a landing aircraft and using that energy for electrical taxiing.*

Authorized licensed use limited to: ULAKBIM UASL - IZMIR YUKSEK TEKNOLOJI ENSTITUSU. Downloaded on February 01,2026 at 23:53:35 UTC from IEEE Xplore. Restrictions apply.